

Estructuras de Datos Avanzadas

Banco nuevo de ejercicios tipo examen

Implementacion - Grafos - Arboles - Iteradores - Casos de uso

Objetivo	Recopilar ejercicios nuevos de complejidad media-alta y alta, inspirados en el estilo de los PDFs de examen: firmas cerradas, comportamiento preciso, excepciones y pruebas unitarias esperadas.
Uso	Sirve como enunciado base para crear nuevos proyectos Maven con TODOs, sin incluir soluciones.
Temario	Grafos, arboles generales, arboles binarios, iteradores y casos de uso con seleccion de estructuras de datos.

Indice

1. Instrucciones generales
2. Implementacion: GraphOperationsExtra
3. Implementacion: BinaryTreeOperationsExtra
4. Implementacion: GeneralTreeOperationsExtra
5. Iteradores de arboles
6. Casos de uso nuevos
7. Anexo: criterios de correccion sugeridos

1. Instrucciones generales

Este documento propone ejercicios nuevos para añadir al repositorio. Están redactados como enunciados de examen: no contienen soluciones, no asumen cambios en las cabeceras publicas y fuerzan a trabajar con estructuras de datos, recorridos y casos frontera.

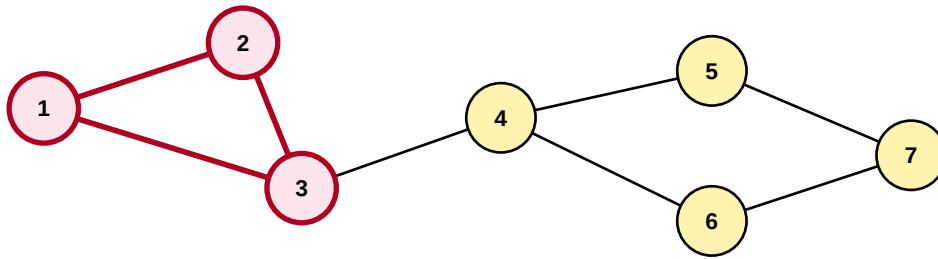
- No se pueden modificar las cabeceras publicas dadas en los esqueletos.
- Solo se pueden añadir atributos, metodos auxiliares o clases privadas cuando el enunciado lo permita.
- Las excepciones indicadas deben ser exactamente del tipo especificado y, cuando se indique mensaje, usar el mismo texto.
- En los ejercicios de grafos se asume grafo no dirigido y sin bucles salvo que se diga lo contrario.
- En los ejercicios de arboles se debe respetar la interfaz Position y no desencapsular nodos internos.
- Los iteradores deben mantener estado coherente despues de llamadas consecutivas a next(), hasNext() y remove(), si existe remove().

Recomendacion de organizacion

Una forma limpia de añadirlos al repositorio seria crear proyectos separados: uno para GraphOperationsExtra, uno para BinaryTreeOperationsExtra, uno para GeneralTreeOperationsExtra, varios proyectos de iteradores, y un bloque de casos de uso limpios.

2. Implementacion: GraphOperationsExtra

Asumir que todos los grafos son no dirigidos, simples y sin bucles. La respuesta debe obtenerse adaptando recorridos de BFS o DFS cuando sea necesario, no lanzando directamente metodos ya resueltos de una clase de algoritmos salvo que el enunciado del proyecto lo permita.



Recorrido recomendado para `shortestCycleLength`: BFS desde cada vertice, evitando reutilizar la arista padre.

Para `distanceClosure`, desde cada origen se conservan solo vertices a distancia menor o igual que `k`.

Figura 1. Grafo de ejemplo: el ciclo mas corto resaltado tiene longitud 3.

componentesConTamanoMinimo (3 puntos)

```
public static <V,E> int componentesConTamanoMinimo(AdjacencyMapGraph<V,E> g, int k)
```

Devuelve cuantas componentes conexas del grafo tienen tamano mayor o igual que `k`. Una componente aislada formada por un unico vertice cuenta como componente de tamano 1.

- Si `g` es null o `k < 1`, lanza `IllegalArgumentException`.
- Si el grafo esta vacio, devuelve 0.
- Debe visitar cada vertice como maximo una vez en el recorrido principal.

Casos que deberian probarse: grafo vacio, vertices aislados, una unica componente, varias componentes y `k` mayor que cualquier componente.

isTree (2 puntos)

```
public static <V,E> boolean isTree(AdjacencyMapGraph<V,E> g)
```

Comprueba si el grafo es un arbol. Un grafo no dirigido es arbol si es conexo y no contiene ciclos.

- Si `g` es null, lanza `IllegalArgumentException`.
- El grafo vacio no se considera arbol.
- Un grafo con un unico vertice y sin aristas si se considera arbol.

Casos que deberian probarse: un vertice, cadena, estrella, grafo con ciclo, grafo no conexo.

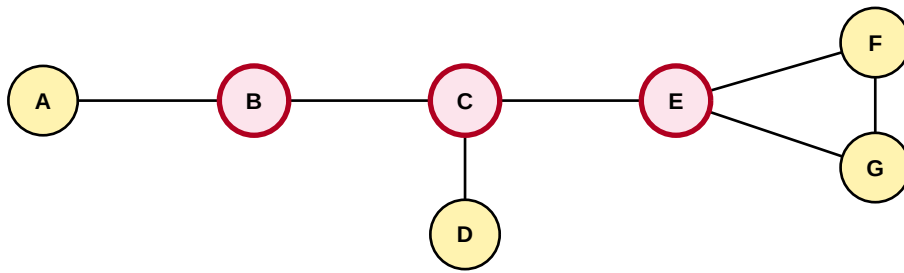
shortestCycleLength (4 puntos)

```
public static <V,E> int shortestCycleLength(AdjacencyMapGraph<V,E> g)
```

Devuelve la longitud del ciclo simple mas corto del grafo. La longitud de un ciclo es el numero de aristas que contiene. Si el grafo no tiene ciclos, devuelve -1.

- Si `g` es null, lanza `IllegalArgumentException`.
- Si el grafo esta vacio o es un arbol, devuelve -1.
- Debe evitar enumerar todos los caminos simples.

Casos que deberian probarse: triangulo, cuadrado, grafo con varios ciclos, grafo aciclico, grafo no conexo con ciclo en una componente.



Version sencilla de examen: eliminar virtualmente cada vertice y repetir un recorrido para contar componentes.

Figura 2. Ejemplo para `verticesQueDesconectan`: B, C y E son vertices criticos.

verticesQueDesconectan (4 puntos)

```
public static <V,E> Collection<Vertex<V>> verticesQueDesconectan(AdjacencyMapGraph<V,E> g)
```

Devuelve los vertices cuya eliminacion aumenta el numero de componentes conexas del grafo. No es necesario eliminar fisicamente el vertice: puede ignorarse durante los recorridos.

- Si g es null, lanza `IllegalArgumentException`.
- Si el grafo tiene 0, 1 o 2 vertices, devuelve una coleccion vacia.
- El orden de salida no es relevante salvo que los tests indiquen lo contrario.

Casos que deberian probarse: grafo cadena, estrella, ciclo, grafo no conexo y grafo con un vertice aislado.

bipartitionGroups (3 puntos)

```
public static <V,E> Pair<Set<Vertex<V>>, Set<Vertex<V>>> bipartitionGroups(AdjacencyMapGraph<V,E> g)
```

Comprueba si el grafo es bipartito mediante coloreado por niveles. Si lo es, devuelve los dos conjuntos de la biparticion. Si no lo es, devuelve null.

- Si g es null, lanza `IllegalArgumentException`.
- Un grafo vacio devuelve dos conjuntos vacios.
- Debe funcionar aunque el grafo no sea conexo.

Casos que deberian probarse: camino par, camino impar, ciclo par, ciclo impar, varios componentes.

distanceClosure (4 puntos)

```
public static <V,E> AdjacencyMapGraph<V,Integer> distanceClosure(AdjacencyMapGraph<V,E> g, int k)
```

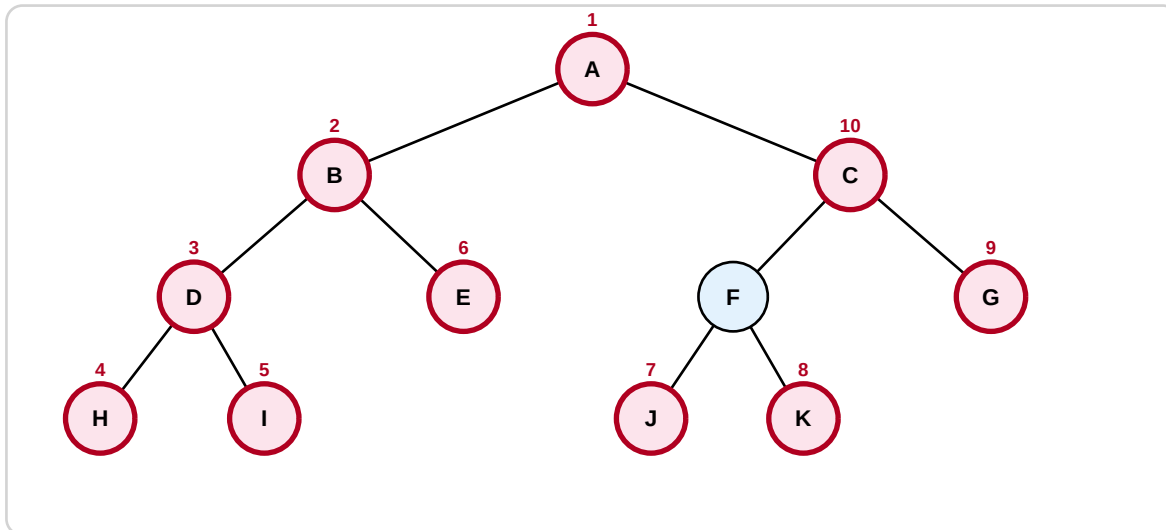
Construye un nuevo grafo con los mismos valores de vertices. En el nuevo grafo hay una arista entre u y v si la distancia minima entre ellos en el grafo original es menor o igual que k. El valor de la arista sera esa distancia minima.

- Si g es null o $k < 1$, lanza `IllegalArgumentException`.
- No deben crearse bucles.
- Si ya existe una arista por una distancia menor, no se reemplaza por una mayor.

Casos que deberian probarse: $k=1$ debe reproducir las aristas originales con valor 1, k grande sobre grafo conexo, grafo no conexo.

3. Implementacion: BinaryTreeOperationsExtra

Estos ejercicios se centran en recorridos, comparacion estructural y borrado seguro en arboles binarios. Cuando haya que eliminar nodos, se recomienda calcular primero las posiciones afectadas y borrar despues, para no invalidar el recorrido.



La frontera no debe repetir nodos aunque un nodo sea borde y hoja a la vez.

Figura 3. Ejemplo de *boundaryTraversal*: A, B, D, H, I, E, J, K, G, C.

removeLeavesAtDepth (3 puntos)

```
public static <E> int removeLeavesAtDepth(BinaryTree<E> tree, int depth)
```

Elimina todas las hojas que se encuentren exactamente a la profundidad indicada. La raíz esta a profundidad 0. Devuelve el numero de nodos eliminados.

- Si tree es null o depth < 0, lanza `IllegalArgumentException`.
- Si el arbol esta vacio, devuelve 0.
- Solo pueden eliminarse nodos que sean hoja en el momento previo al borrado.

Casos que deberian probarse: arbol vacio, raíz unica con depth 0, hojas a distintos niveles, depth mayor que la altura.

boundaryTraversal (4 puntos)

```
public static <E> List<E> boundaryTraversal(BinaryTree<E> tree)
```

Devuelve los elementos de la frontera exterior del arbol: raíz, borde izquierdo sin hojas, todas las hojas de izquierda a derecha y borde derecho sin hojas en orden inverso.

- Si tree es null, lanza `IllegalArgumentException`.
- Si tree esta vacio, devuelve una lista vacia.
- No puede devolver elementos repetidos.

Casos que deberian probarse: arbol degenerado a la izquierda, degenerado a la derecha, arbol completo, arbol con una sola raíz.

areMirror (3 puntos)

```
public static <E> boolean areMirror(BinaryTree<E> a, BinaryTree<E> b)
```

Comprueba si dos arboles binarios son espejo. Dos arboles son espejo si sus raíces contienen el mismo elemento y el hijo izquierdo de uno es espejo del hijo derecho del otro, y viceversa.

- Si alguno de los arboles es null, lanza `IllegalArgumentException`.
- Dos arboles vacios son espejo.
- Debe comparar estructura y elementos.

Casos que deberian probarse: dos vacios, uno vacio y otro no, arbol simetrico, mismo arbol no espejo.

isMinHeap (4 puntos)

```
public static <E> boolean isMinHeap(BinaryTree<E> tree, Comparator<E> comparator)
```

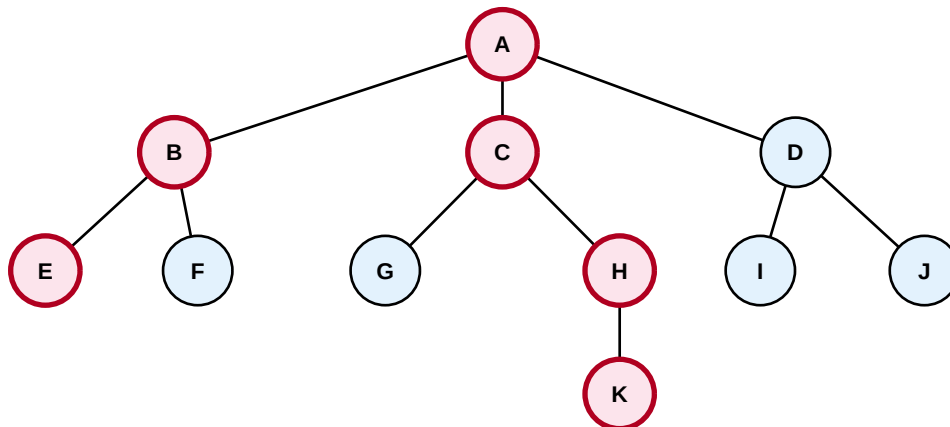
Comprueba si el arbol binario cumple las propiedades de un monticulo minimo: es casi completo y todo padre tiene prioridad menor o igual que la de sus hijos segun el comparador.

- Si tree o comparator son null, lanza IllegalArgumentException.
- Un arbol vacio cumple la propiedad.
- La comprobacion de completitud debe hacerse por niveles.

Casos que deberian probarse: arbol casi completo valido, arbol con hueco a la izquierda, prioridad padre-hijo incorrecta, duplicados permitidos.

4. Implementacion: GeneralTreeOperationsExtra

Los siguientes ejercicios trabajan con arboles generales. Son utiles para practicar children(), parent(), isLeaf(), isRoot() y recorridos por niveles o postorden.



Para calcular diametro, en cada nodo interesan las dos mayores alturas de sus hijos.

Para pathToRoot, basta seguir parent() hasta llegar a la raiz.

Figura 4. Ejemplo de arbol general: un diametro posible es E-B-A-C-H-K, de longitud 5.

removeSubtreesSmallerThan (4 puntos)

```
public static <E> int removeSubtreesSmallerThan(NAryTree<E> tree, int k)
```

Elimina todos los subarboles propios cuyo tamaño sea menor que k. El tamaño de un subarbol es el número de nodos que contiene. Devuelve el número total de nodos eliminados.

- Si tree es null o $k < 1$, lanza IllegalArgumentException.
- La raíz no se elimina aunque su subarbol sea menor que k.
- Si se elimina un subarbol, no se siguen evaluando sus descendientes por separado.

Casos que deberian probarse: subarboles hoja, subarboles encadenados, $k=1$, k mayor que el tamaño del arbol.

pathToRoot (2 puntos)

```
public static <E> List<E> pathToRoot(Tree<E> tree, Position<E> p)
```

Devuelve los elementos desde la posición p hasta la raíz, ambos incluidos. El primer elemento de la lista debe ser el de p.

- Si tree o p son null, lanza IllegalArgumentException.
- Si p no pertenece al arbol, debe lanzarse IllegalArgumentException si la implementacion permite detectarlo.

Casos que deberian probarse: p es raíz, p es hoja profunda, p invalida.

nodesAtDistanceFromRoot (3 puntos)

```
public static <E> List<Position<E>> nodesAtDistanceFromRoot(Tree<E> tree, int k)
```

Devuelve todas las posiciones situadas a profundidad k. La raíz esta a profundidad 0.

- Si tree es null o $k < 0$, lanza IllegalArgumentException.
- Si tree esta vacio, devuelve lista vacia.
- El orden debe ser de izquierda a derecha segun children().

Casos que deberian probarse: $k=0$, k intermedio, k mayor que altura, nodos con distinto número de hijos.

sameShape (3 puntos)

```
public static boolean sameShape(Tree<?> a, Tree<?> b)
```

Comprueba si dos arboles generales tienen exactamente la misma forma, ignorando los elementos almacenados. Dos nodos coinciden en forma si tienen el mismo numero de hijos y sus hijos coinciden en el mismo orden.

- Si alguno de los arboles es null, lanza `IllegalArgumentException`.
- Dos arboles vacios tienen la misma forma.

Casos que deberian probarse: mismo numero de nodos pero distinta organizacion, mismo arbol con valores distintos, hijos en orden distinto.

diameter (4 puntos)

```
public static <E> int diameter(Tree<E> tree)
```

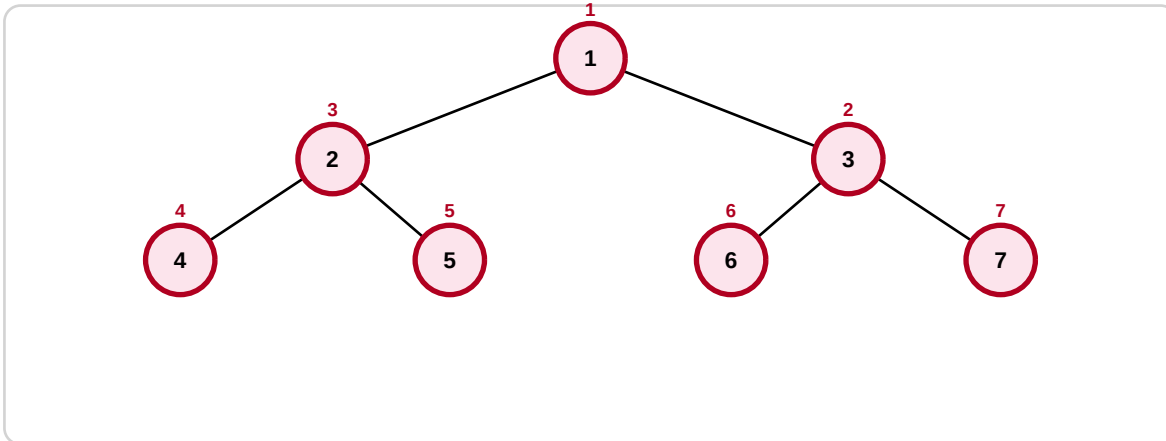
Calcula la mayor distancia entre dos nodos del arbol. La distancia se mide en numero de aristas. Un arbol vacio o con un unico nodo tiene diametro 0.

- Si tree es null, lanza `IllegalArgumentException`.
- Debe resolverse sin recalculiar la altura completa desde cada nodo.

Casos que deberian probarse: cadena, estrella, arbol equilibrado, arbol con diametro que no pasa por la raiz.

5. Iteradores de arboles

Cada iterador debe implementar correctamente `hasNext()` y `next()`. Cuando se pida `remove()`, debe seguir la semantica habitual: solo se puede llamar una vez por cada `next()` correcto y debe eliminar el ultimo elemento devuelto.



Nivel 0: izquierda a derecha. Nivel 1: derecha a izquierda. Nivel 2: izquierda a derecha.

Figura 5. Zigzag por niveles: 1, 3, 2, 4, 5, 6, 7.

ZigZagBinaryTreeIterator (3 puntos)

```
public class ZigZagBinaryTreeIterator<E> implements Iterator<Position<E>>
```

Itera un árbol binario por niveles alternando el sentido de cada nivel. El primer nivel se recorre de izquierda a derecha.

- El constructor recibe un `BinaryTree<E>`.
- Si el árbol es null, lanza `IllegalArgumentException`.
- Si el árbol está vacío, `hasNext()` devuelve false.

Casos que deberían probarse: árbol completo, árbol incompleto, árbol con un único nodo.

PostorderLeafIterator (2 puntos)

```
public class PostorderLeafIterator<E> implements Iterator<Position<E>>
```

Itera únicamente las hojas de un árbol binario siguiendo el orden postorden.

- No debe devolver nodos internos.
- `next()` sin elementos pendientes lanza `NoSuchElementException`.

Casos que deberían probarse: solo raíz, árbol sin hijo derecho, árbol completo.

InternalLevelIterator (2 puntos)

```
public class InternalLevelIterator<E> implements Iterator<Position<E>>
```

Recorre por anchura solo los nodos internos de un árbol general o binario, según el proyecto donde se ubique.

- Una hoja nunca debe aparecer en la salida.
- La raíz aparece si no es hoja.

Casos que deberían probarse: árbol vacío, solo raíz, varios niveles.

RightViewIterator (3 puntos)

```
public class RightViewIterator<E> implements Iterator<Position<E>>
```

Devuelve el nodo visible desde la derecha en cada nivel de un árbol binario. Equivale al último nodo visitado en cada nivel de un BFS normal de izquierda a derecha.

- Debe devolver como máximo un nodo por nivel.
- En árbol vacío no hay elementos.

Casos que deberían probarse: árbol completo, ramas solo izquierdas, ramas solo derechas, huecos intermedios.

LevelRangeIterator (3 puntos)

```
public class LevelRangeIterator<E> implements Iterator<Position<E>>
```

Itera los nodos cuyas profundidades estan entre minLevel y maxLevel, ambos incluidos, usando orden por niveles.

- Si minLevel < 0 o maxLevel < minLevel, lanza IllegalArgumentException.
- No debe almacenar nodos de niveles fuera del rango salvo como apoyo minimo para llegar al rango.

Casos que deberian probarse: rango que empieza en 0, rango fuera de la altura, rango de un solo nivel.

PostorderRemovelerator (4 puntos)

```
public class PostorderRemoveIterator<E> implements Iterator<Position<E>>
```

Itera en postorden sobre un arbol binario y permite remove(). remove() elimina del arbol el ultimo nodo devuelto por next() usando el metodo remove del arbol.

- remove() antes de next() lanza IllegalStateException.
- Dos remove() consecutivos sin next() entre medias lanzan IllegalStateException.
- Tras borrar, el iterador debe continuar con los nodos pendientes que sigan existiendo.

Casos que deberian probarse: borrar hojas, borrar nodo interno despues de sus hijos, borrar todos los nodos uno a uno.

AncestorIterator (2 puntos)

```
public class AncestorIterator<E> implements Iterator<Position<E>>
```

Dada una posicion inicial, itera sus ancestros: padre, abuelo, bisabuelo, hasta llegar a la raiz. La posicion inicial no se devuelve.

- Si la posicion inicial es la raiz, el iterador esta vacio.
- Si la posicion es null, lanza IllegalArgumentException.

Casos que deberian probarse: posicion hoja, posicion interna, raiz.

GeneralTreeZigZagIterator (4 puntos)

```
public class GeneralTreeZigZagIterator<E> implements Iterator<Position<E>>
```

Version para arbol general del recorrido zigzag por niveles. En niveles de derecha a izquierda, los hijos deben considerarse en el orden inverso al iterable children().

- Debe respetar el orden de children() en niveles pares.
- Debe invertir el orden por nivel, no solo el orden de insercion en la cola.

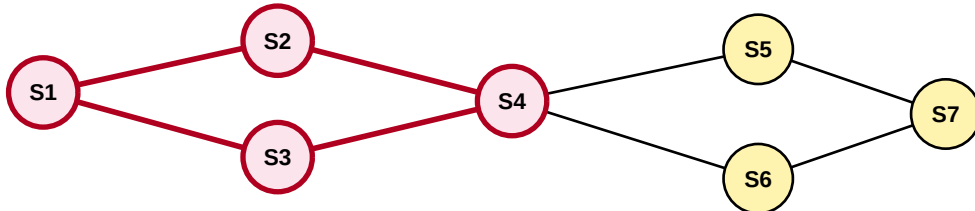
Casos que deberian probarse: nodos con muchos hijos, niveles con un solo nodo, arbol no equilibrado.

6. Casos de uso nuevos

Estos casos estan pensados para la parte de seleccion de estructuras de datos: clases de dominio dadas, una clase gestora vacia y metodos publicos cerrados. La dificultad esta en mantener indices coherentes, no en reimplementar las estructuras.

Caso 1. RedSensoresIncendios

Dificultad	Muy alta	Sensores conectados, alertas de incendio, difusion limitada y limpieza de alertas antiguas.
------------	----------	---



El caso combina mapa de sensores, grafo de conexiones y alertas ordenadas por fecha.

Figura 6. Difusion desde S1 con maxSaltos = 2: alcanzados S1, S2, S3 y S4.

- Clases dadas: Sensor(id, zona, bateria), Alerta(id, sensorOrigen, instante, gravedad, mensaje).
- En RedSensoresIncendios se deben elegir estructuras privadas para sensores, conexiones y memoria de alertas.
- public boolean addSensor(Sensor sensor): Añade un sensor si su id no existe.
- public boolean conectar(String id1, String id2): Crea una conexion no dirigida entre sensores existentes.
- public Set<String> difundirAlerta(Alerta alerta, int maxSaltos): Registra y propaga una alerta hasta maxSaltos conexiones. Devuelve ids alcanzados.
- public int borrarAlertasHasta(LocalDateTime instante): Elimina alertas con fecha menor o igual al instante.
- public String sensorMasCentral(): Devuelve el sensor que minimiza la distancia maxima al resto; si la red no es conexas devuelve null.

Caso 2. GestorExamenesEDA

Dificultad	Alta	Alumnos, ejercicios, entregas, notas, revision y ranking por convocatoria.
------------	------	--

- Clases dadas: Alumno(dni, nombre, grupo), Ejercicio(codigo, bloque, puntosMaximos), Entrega(dni, codigoEjercicio, fecha, nota, estado).
- public boolean addAlumno(Alumno alumno): Evita duplicados por DNI.
- public boolean addEjercicio(Ejercicio ejercicio): Evita duplicados por codigo.
- public boolean registrarEntrega(Entrega entrega): Solo permite una entrega activa por alumno y ejercicio.
- public List<Entrega> entregasDeAlumno(String dni): Devuelve entregas del alumno ordenadas por fecha.
- public List<Alumno> ranking(int n): Devuelve los n alumnos con mayor nota total.
- public List<Entrega> entregasEntreNotas(double min, double max): Consulta por rango de notas, extremos incluidos.

Caso 3. HospitalUrgenciasAvanzado

Dificultad	Muy alta	Pacientes, triaje, boxes, cancelaciones, historial y consultas por fecha.
------------	----------	---

- Clases dadas: Paciente(id, nombre, edad), Episodio(codigo, pacienteId, gravedad, llegada, estado), Box(id, especialidad, disponible).
- public boolean registrarPaciente(Paciente paciente): Da de alta un paciente por id.
- public boolean registrarEpisodio(Episodio episodio): Registra un episodio pendiente de atencion.
- public Episodio atenderSiguiente(): Atiende el episodio de mayor gravedad y, a igualdad, mas antiguo.
- public boolean cancelar(String codigo): Cancela un episodio pendiente.
- public List<Episodio> episodiosAntesDe(LocalDateTime fecha): Devuelve pendientes con llegada menor o igual a fecha.
- public String pacienteMasAtendido(): Devuelve el id del paciente con mas episodios no cancelados.

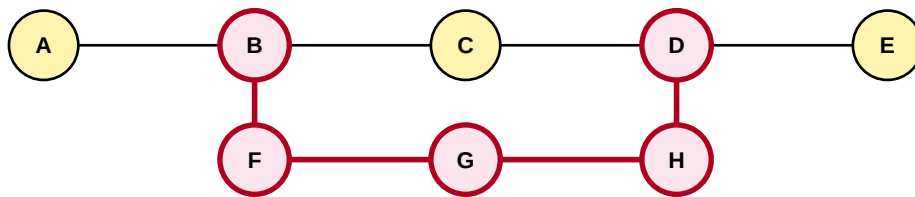
Caso 4. BibliotecaReservasV2

Dificultad	Muy alta	Libros, ejemplares, prestamos, reservas y vencimientos.
------------	----------	---

- Clases dadas: Libro(isbn, titulo, autor), Ejemplar(codigo, isbn), Prestamo(codigo, usuario, ejemplar, fechaInicio, fechaFin), Reserva(codigo, usuario, isbn, fecha).
- public boolean addLibro(Libro libro): Registra libro sin duplicar ISBN.
- public boolean addEjemplar(Ejemplar ejemplar): Asocia ejemplar a un libro existente.
- public Prestamo prestar(String usuario, String isbn, LocalDateTime fecha): Presta ejemplar disponible o crea reserva si no hay disponibles.
- public boolean devolver(String codigoPrestamo, LocalDateTime fecha): Devuelve ejemplar y atiende la primera reserva pendiente del ISBN.
- public List<Prestamo> vencidosHasta(LocalDateTime fecha): Prestamos activos vencidos hasta fecha.
- public List<Libro> topPrestados(int n): Libros mas prestados historicamente.

Caso 5. RedTransportesURJC

Dificultad	Muy alta	Paradas, líneas, conexiones y trayectos con máximo de saltos o transbordos.
------------	----------	---



El método `existeTrayecto` debe poder limitar por número de saltos.

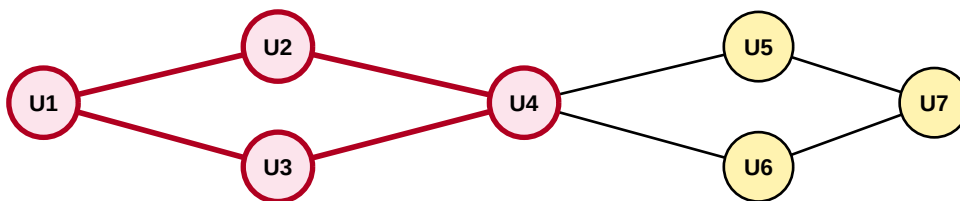
El método `parMasConectada` se calcula por grado o salidas registradas, según el enunciado del método.

Figura 7. Trayecto alternativo B-F-G-H-D con 4 aristas.

- Clases dadas: `Parada(id, nombre, zona)`, `Conexion(origen, destino, linea, minutos)`.
- `public boolean addParada(Parada parada)`: Añade parada sin duplicar id.
- `public boolean addConexion(String origen, String destino, String linea, int minutos)`: Añade conexión no dirigida entre paradas existentes.
- `public boolean existeTrayecto(String origen, String destino, int maxSaltos)`: Comprueba si hay camino con longitud menor o igual que `maxSaltos`.
- `public List<String> paradasAlcanzables(String origen, int maxSaltos)`: Devuelve ids de paradas alcanzables por niveles.
- `public String paradaMasConectada()`: Devuelve la parada con mayor número de conexiones.
- `public int cerrarZona(String zona)`: Elimina o desactiva las paradas de una zona y devuelve cuantas se cerraron.

Caso 6. PlataformaMensajes

Dificultad	Alta	Usuarios, relaciones de confianza, mensajes de difusión y limpieza por fecha.
------------	------	---



Parecido al caso de routers: hay que recordar mensajes recibidos para evitar duplicados.

Figura 8. Difusión desde U1 en dos saltos.

- Clases dadas: `Usuario(id, nombre)`, `Mensaje(id, emisor, fecha, contenido, difusión)`.
- `public boolean addUsuario(Usuario usuario)`: Registra usuario por id.
- `public boolean addRelacion(String id1, String id2)`: Relación no dirigida de confianza.
- `public boolean recibirMensaje(String usuarioId, Mensaje mensaje)`: Devuelve true si ese usuario no había recibido antes el mensaje.

- public int borrarMensajesHasta(LocalDateTime fecha): Elimina de todas las memorias mensajes antiguos.
- public Set<String> difundir(String origen, Mensaje mensaje): Devuelve usuarios que reciben el mensaje por primera vez.
- public String usuarioCentral(): Nodo que minimiza la distancia maxima al resto; null si la red no es conexas.

Caso 7. ControlEventosAforo

Dificultad	Alta	Eventos, entradas, accesos por QR, zonas y expulsiones.
------------	------	---

- Clases dadas: Evento(id, nombre, fecha, aforoMaximo), Entrada(qr, dni, eventold, zona), Acceso(qr, instante, puerta, tipo).
- public boolean crearEvento(Evento evento): Crea evento por id.
- public boolean emitirEntrada(Entrada entrada): Emite entrada si no supera aforo y QR no existe.
- public boolean registrarAcceso(String qr, LocalDateTime instante): Permite acceso si la entrada existe y no ha entrado antes.
- public boolean expulsar(String qr, LocalDateTime instante): Registra salida forzada si la persona esta dentro.
- public int aforoActual(String eventold): Numero de asistentes dentro.
- public List<Entrada> entradasPorZona(String eventold, String zona): Consulta entradas de una zona.

Caso 8. SistemaPaquetesPrioritarios

Dificultad	Muy alta	Paquetes, almacenes, prioridad, fecha limite y asignacion de salida.
------------	----------	--

- Clases dadas: Paquete(localizador, destino, prioridad, peso, fechaLimite), Almacen(id, zona, capacidad), Salida(codigo, almacen, destino, fecha).
- public boolean registrarPaquete(Paquete paquete): Registra paquete sin duplicar localizador.
- public boolean registrarAlmacen(Almacen almacen): Registra almacen.
- public Paquete siguientePaquete(): Devuelve el pendiente mas prioritario; empate por fecha limite mas antigua.
- public boolean reprogramar(String localizador, LocalDateTime nuevaFecha): Actualiza la fecha y reordena indices.
- public List<Paquete> vencidosHasta(LocalDateTime fecha): Pendientes con fecha limite vencida.
- public int eliminarDestino(String destino): Elimina todos los paquetes pendientes de un destino.
- public String destinoMasUrgente(): Destino con mas paquetes de prioridad alta pendientes.

7. Anexo: criterios de correccion sugeridos

Para convertir estos en proyectos del repositorio, se recomienda mantener el mismo patron: esqueletos con UnsupportedOperationException, tests publicos de contrato y tests ocultos o avanzados para invariantes.

Bloque	Criterios recomendados
Grafos	Validar nulos, vacios, no conexos, vertices aislados, ciclos pequenos, varios componentes y que no se llamen directamente algoritmos prohibidos.
Arboles binarios	Probar raiz unica, arbol degenerado, arbol completo, huecos, duplicados y borrados que no rompan posiciones pendientes.
Arboles generales	Probar distinto numero de hijos, orden de children(), subarboles pequenos, diametro que no pasa por la raiz y pathToRoot desde raiz/hoja.
Iteradores	Probar hasNext() repetido, next() al final, arbol vacio, un nodo, recorridos incompletos y remove() ilegal cuando aplique.
Casos de uso	Probar duplicados, altas/bajas, indices cruzados, actualizaciones que cambian el orden, rangos de fechas y consultas tras borrar.

Seleccion prioritaria para empezar

- IMPLEMENTACION/11_grafos_ciclo_mas_corto
- IMPLEMENTACION/12_grafos_vertices_criticos_basico
- IMPLEMENTACION/16_arbol_binario_frontera
- IMPLEMENTACION/19_arbol_general_podar_subarboles
- IMPLEMENTACION/23_arbol_general_diametro
- ITERADORES/18_postorder_remove_iterator
- CASOS_DE_USO/LIMPIOS/08_red_sensores_incendios
- CASOS_DE_USO/LIMPIOS/12_red_transportes_urjc
- CASOS_DE_USO/LIMPIOS/13_plataforma_mensajes